JUL 17

Design ChatGPT Tasks

假設我們已經有 **ChatGPT custom connector（自訂連接器）**，請設計一個系統，能夠支援 LLM ChatBot 在指定時間排程執行 jobs。

這些 jobs 可以是 **一次性（one-time）** 或 **週期性（periodic）**，類似於 ChatGPT Tasks。

功能性需求（Functional Requirements）

1. 使用者可以預約現在或未來的 jobs，且 jobs 可以是一次性或重複執行（one-time / recurring）。
2. 使用者可以檢視已預約 jobs 的狀態（job status）。
3. 使用者可以透過自然語言（natural language）與 ChatBot 互動來預約 jobs。
    - 例如：「每天早上八點幫我總結財經新聞」

非功能性需求（Non-Functional Requirements）

1. 系統必須具備高可用性（high availability）。
2. 可擴展性（scalability）：系統需支援最高 10K jobs / second 的排程請求。
3. 必須提供至少一次執行保證（at-least-once job execution guarantee）。

API 設計（API Design）

1. 排程工作（Schedule a job）

```
POST v1/jobs
{
  action,
  job_params: {
```

buildmoat.org

```
    type: immediate | one-time | recurring,
    time,// 僅 one-time 需要，指定未來執行時間
    schedule// 僅 recurring 需要，cron 格式，例如: * 10 * * *
}
```

- `action`（工作行為）：實際要執行的任務類型，例如呼叫某個 connector、產生摘要、發送通知等。
- `job_params`（工作參數）：
  - `type`（工作類型）：
    - `immediate`：立即執行
    - `one-time`：單次排程
    - `recurring`：週期性排程
  - `time`（執行時間）：指定未來的執行時間，僅適用於 one-time。
  - `schedule`（排程規則）：cron 格式的排程表達式，僅適用於 recurring。

2. 查詢工作狀態（View job status）

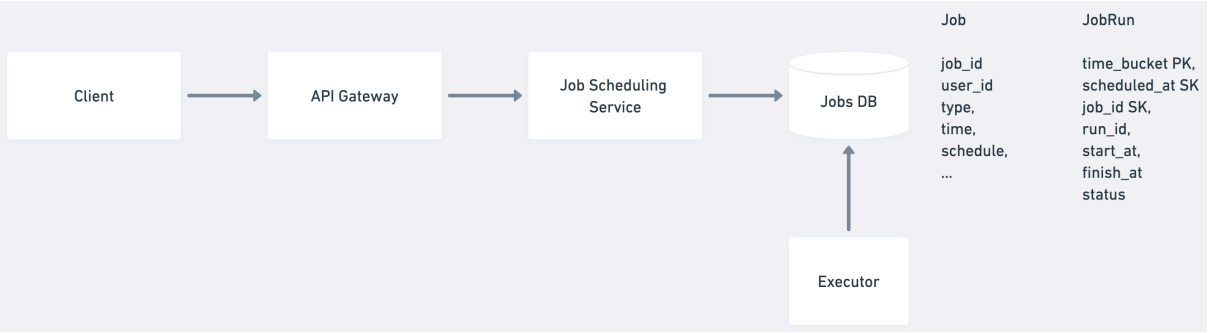
```
# 依狀態列出使用者的 jobs
GET /jobs?status={status}&start_time={start_time}&end_time={end_time}&pageSize={page_size}&page={page_number}

→ Job[]
```

- `status`（工作狀態）：例如 scheduled、running、completed、failed。
- `start_time` / `end_time`：依時間範圍篩選 jobs。
- `pageSize` / `page`：分頁參數，用於大量 jobs 查詢。
- 回傳結果為 Job 清單。

High-Level Design

1. Job 排程流程（Job Scheduling Flow）

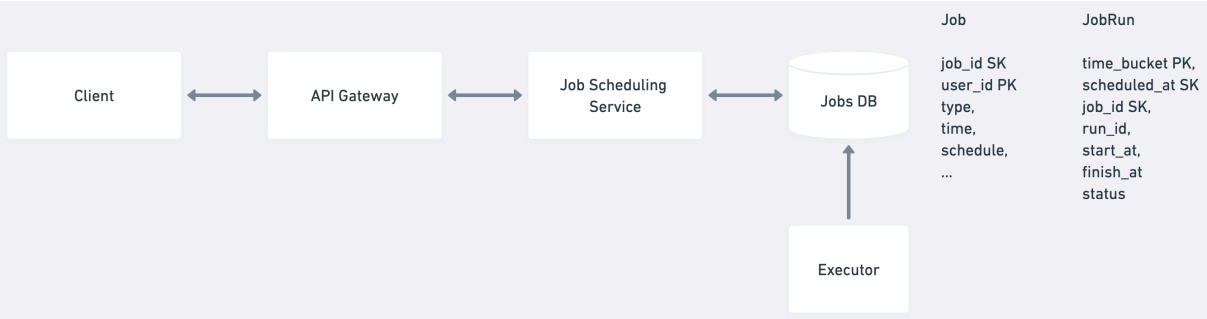
buildmoat.org



1. 使用者向 Job Scheduling service（工作排程服務）送出排程請求。
2. Job Scheduling service 將 Job metadata（工作中繼資料）寫入 Jobs table，並在 JobRun table 中建立一筆對應的 job run metadata，初始狀態為 `PENDING`。
   a. 由於需要支援 recurring jobs（週期性工作），單一 job 會對應多個未來的 job runs，因此需要獨立的 JobRun table 來表示每一次實際執行實體。
3. Executor（執行器）會定期掃描 JobRun table，找出即將到期且狀態為 `PENDING` 的 job runs（例如 `scheduled_at` 落在接下來 5 分鐘內），並在正確時間點執行。
4. Job run 結束後，executor 會將狀態更新為 `SUCCEEDED`。
   a. 若該 job 為 recurring，executor 會計算下一次執行時間，並在 JobRun table 中寫入新的 job run。
   b. 若 job run 執行失敗，executor 會將狀態更新為 `RETRYING` 並進行重試；當所有 retries 都耗盡後，狀態更新為 `FAILED`。
5. 由於寫入吞吐量可能高達 10K writes/sec，可選擇使用 NoSQL（例如 DynamoDB 或 Cassandra）。
6. 為了避免 broadcast reads（掃描所有 partitions），我們引入 `time_bucket` 欄位：
   - `time_bucket` 由 `scheduled_at` truncate 至最近一小時形成，並作為 partition key。
   - (`scheduled_at`, `job_id`) 作為 sorting key，使同一 partition 內依執行時間排序。
   - 這樣在定位特定時間區間的 job runs 時，時間複雜度可接近 O(log n)。
   - 若有 hot shard 的疑慮，可在 partition key 後加 shard suffix，例如 `YYYYMMDDHH#S{00..31}`，以分散讀寫負載。

2. Job 檢視流程（Job Viewing Flow）

buildmoat.org



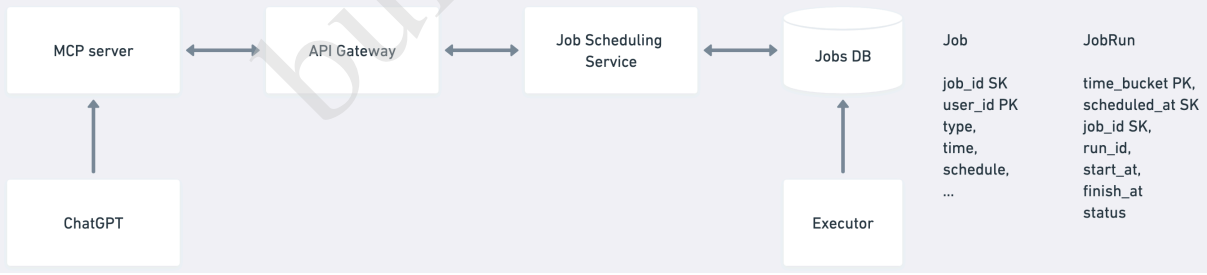
從使用者體驗（UX）角度來看，讀取模式主要有兩種：

1. 使用者列出自己所有已排程的 jobs，系統依建立時間排序回傳。
2. 使用者從清單中選擇某一個 job，並進行檢視或編輯。

為了支援上述操作，我們可採用以下資料模型：

- 使用 `user_id` 作為 partition key。
- 使用 `job_id` 作為 sorting key（前提是 `job_id` 為 time-sortable，例如 ULID / KSUID）。
- 若 `job_id` 無法保證時間排序，則可建立 Global Secondary Index（GSI），
  - PK：`user_id`
  - SK：`created_at`

3. MCP Server



當 Job Scheduler service 準備好後，下一步是透過 custom connector 與 ChatGPT 整合。

此時需要引入 **MCP server（Model Context Protocol server，模型上下文協定伺服器）** 作為中介。

MCP（Model Context Protocol） 是一個開源標準，用於讓 AI 應用（例如 ChatGPT、Claude）連接外部系統，包括：

- 資料來源（local files、databases）

buildmoat.org

- 工具（search engines、calculators）
- Prompts（system prompts、few-shot examples）

與 ChatGPT 的整合流程如下：

1. **Discovery（探索，一個 thread 只做一次）**

   ChatGPT 連線至 MCP server，呼叫 `tools/list`。

   回傳的 tool metadata（名稱、描述、JSON Schema）會被快取在該 thread 中，作為 `mcp_list_tools`，避免每一輪對話重複查詢。

2. **每一輪模型可見的輸入（Inputs the model sees each turn）**

   每次使用者輸入時，model 會看到：

   - system instructions
   - user message
   - 快取的 MCP tool list

     （以及你設定的限制，例如 `allowed_tools`、`tool_choice`）

3. **Tool selection（工具選擇）**

   Model 根據 tool 的名稱、描述與 JSON Schema，判斷是否有適合的 tool（例如「建立任務」、「查詢排程」）。

4. **Argument filling & invocation（參數填充與呼叫）**

   若決定使用某個 tool，model 會依 schema 組出 JSON args，並產生 `mcp_tool_call`。

   Runtime 會將該呼叫送至 MCP server，並等待 `mcp_tool_result`。

5. **MCP server 轉發請求**

   MCP server 將 tool call 轉發至實際的 Job Scheduler API，並回傳乾淨的 JSON 結果。

6. **自然語言回覆（Final response）**

   Model 將 tool 回傳的 JSON 結果整合成自然語言回覆給使用者。

深入探討（Deep Dives）

1. 我們如何確保系統能準時完成 job 執行？

buildmoat.org

目前 executor 同時負責 **查詢（querying）** 與 **執行（job running）**，這在使用者與 jobs 規模放大後會讓系統變得脆弱，原因包括：

1. 若發生失敗或 job 執行時間過長，executor 可能無法準時查詢下一批 jobs。

   例如：executor 需要每 5 分鐘查詢一次 DB，但某個 job 本身執行超過 5 分鐘。

2. 若在短時間內出現 job 執行高峰，executor 可能無法在時間窗內完成所有 jobs。

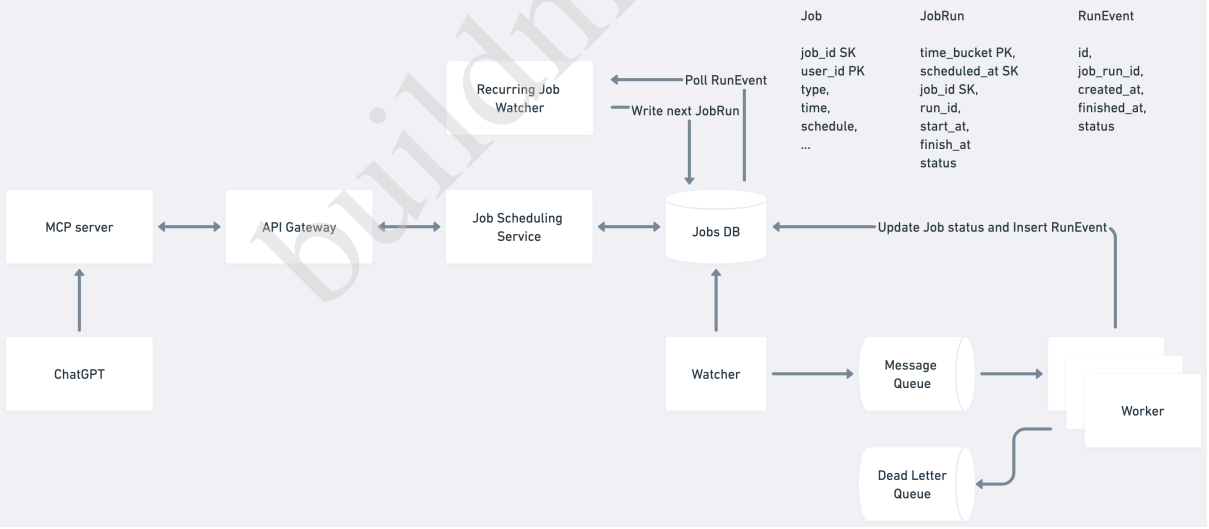
   例如：平時 5 分鐘內約 1000 個 job runs，但在特殊情境（如 Black Friday 提醒）時，大量使用者同時排程。

為了提升系統健壯性，我們需要：

- **分離查詢與執行（separate querying and job running）**
- 引入 **queue（佇列）** 作為 buffer 以吸收突發流量

Queue 的核心目的在於解耦 DB querying 與 job execution：

1. 隔離 job execution failures，不影響 querying 流程。
2. Job execution workers 可獨立於 watcher 水平擴展。
3. Message queue 提供 durability：僅在成功執行後才移除訊息（例如使用 SQS）。



引入 message queue 後，job scheduling 流程調整如下：

1. 使用者向 Job Scheduling service 提交排程請求。
2. Job Scheduling service 將 Job metadata 寫入 Jobs table，並在 JobRun table 中建立狀態為 `PENDING` 的 job run。

buildmoat.org

3. **Watcher process（監看程序）** 定期查詢 JobRun table，找出即將到期且狀態為 `PENDING` 的 job runs（例如 `scheduled_at` 落在接下來 5 分鐘內），並將其發佈到 message queue。
4. **Workers（工作節點）** 消費 queue 中的訊息，根據 DB 中的 JobRun metadata 執行 job；完成後更新狀態為 `SUCCEEDED`。
   a. 若 job 執行失敗，worker 會將狀態更新為 `RETRYING` 並重試；當 retries 用盡後，更新為 `FAILED`，並將 job 發送至 **DLQ（Dead Letter Queue，死信佇列）** 以供後續檢查。
5. Worker 會在同一個 transaction 中：
   - 更新 job 狀態
   - 寫入一筆 **RunEvent（執行事件）** 至 JobsDB（append-only）

   對於 recurring jobs，會有一個獨立的 **RecurringJobWatcher（週期性工作監看器）** 定期輪詢 RunEvent 中已到達 terminal status 的事件，並插入下一次執行的 JobRun。

   1. 使用集中式方式計算與寫入下一次執行時間，將（DB querying + insertion）與 job execution 解耦。
   2. 採用 append-only RunEvent outbox，讓 scheduler 對 immutable events 作出反應，而非輪詢 mutable state，能有效降低 race conditions。

      搭配 transactional claim-and-mark 與 idempotent inserts，可在 crash 或 retry 情境下避免重複排程。

2. 我們如何確保至少一次執行（at-least-once execution）？

在執行階段，workers 會依 retry policy 重試 job，最終失敗的 job 會被送入 DLQ。

但前提是：**失敗必須能被系統觀察到**。

若 worker 在執行期間 crash 並離線，job 可能「悄悄消失（silent failure）」。

為避免這種情況，需要某種 **heartbeat（心跳）** 機制，讓系統能在逾時後判定 job 失敗並交由其他 worker 重試。

幸運的是，Amazon SQS 提供可直接利用的功能：**Visibility Timeout（可見性逾時）**。

運作方式如下：

bundnoat.org

- 當 worker 從 queue 取得訊息後，SQS 會在一段時間內讓該訊息對其他 workers 不可見。
- Worker 需定期延長 visibility timeout（heartbeat）。
- Worker 成功完成 job 後，刪除訊息。
- 若 worker crash 或在 timeout 內未完成處理，SQS 會自動讓訊息重新可見，交由其他 workers 處理。

此機制可有效防止 worker 在 crash 後造成的 silent failures，並協助我們達成 at-least-once execution。

> ⚠️ 在 at-least-once 模式下，job 必須是 idempotent（具冪等性）：
>
> 重複執行多次，其結果必須等同於只執行一次。

3. 我們如何讓 ChatGPT 與 MCP Server 的整合更可靠？

以下是實務上能顯著提升可靠性的設計原則：

1. 使用 **action-verb 命名**（例如 `task.create@v1`），並加上一句「何時使用 / 何時不使用」的描述。
   - Model 在將使用者動詞（fetch / create / cancel / summarize）對應到 tools 時，對 imperative 命名更可靠。
   - 簡短描述可在不增加太多 tokens 的情況下避免歧義。
2. 一律加上 **version suffix（版本後綴）**。
   - Chat 會快取 tool list；在同名 tool 下修改 schema 可能破壞長對話。
   - 新版本用 `@v2`，舊版本 `@v1` 繼續可用。
3. 定義 **嚴謹的 JSON Schema**（`required`、enums、defaults、`additionalProperties:false`）。
   - 嚴謹 schema 可降低錯誤率、減少 retries，並讓 auto-repair 更容易。
   - `required`：僅標示 tool 無法自行推斷的欄位。
   - Enums：用於有限集合的輸入。
   - Defaults：設定合理預設值（例如 `type: "immediate"`）。
   - `additionalProperties:false`：拒絕多餘欄位，讓 model 更快學會正確格式。
4. **加入極簡 system instruction（一次即可）**

buildmoat.org

- 在第一個 system message 中說明何時使用哪些 tool，以及缺少輸入時的預設行為。
- 建議控制在 ~150 tokens 以內。範例：

> “Use news.fetch@v1 when the user requests current articles. If topics or date are missing, default to ['ai','markets'] and today 00:00 local. Ask one clarifying question only if absolutely necessary.”

5. 回傳 **結構化、可修正的錯誤（structured, fixable errors）**，讓 model 能自我修正後重試，例如：

```
{
  "ok": false,
  "error": {
    "code": "USER_INPUT",
    "message": "Invalid 'action'.",
    "field": "action",
    "expected": ["send_email","call_llm"]
  }
}
```

buildmoat.org